

ZZedc Permissions Hierarchy via Google Sheets: Design, Analysis, and Construction

2026-05-28 18:33 PDT

This document covers two related topics. The first is the design and implementation of the Google Sheets change monitor added to ZZedc for the Prazosin Sleep Study (Large), protocol PRAZ-L-2026. The second is an analysis of standard EDC permission hierarchies and a construction guide for building the correct hierarchy in ZZedc using the Google Sheets seed-import pathway.

Each topic is treated in two registers: a technical section using database and systems terminology, followed by a plain-language section for the data scientist and study staff who will operate the system.

Part 1: Change Monitor – Technical Description (Database Expert Register)

Background and motivation

ZZedc supports a Google Sheets workbook as a bootstrap seed source for three configuration relations: the principal registry (`Study_Users`), the field metadata catalogue (`Data_Dictionary`), and the constraint definition table (`validation_rules`). In the multi-site prazosin trial, each site coordinator holds editor-level ACL grants on the `Study_Users` tab of their per-site workbook. The `Data_Dictionary` and `validation_rules` tabs are viewer-only for coordinators, enforced via Google Sheets tab protection. This arrangement allows non-technical site staff to propose changes to their site's user roster without requiring data-scientist mediation for every routine addition.

Without a change-detection layer, the only path from sheet to database was a manual, full-table re-import via `setup_zzedc_from_gsheets()`. That operation writes directly to the live `edc_users` relation with no review gate. A coordinator could add an account with an incorrect role, or modify another user's site scope, and the change would land in the database on the next import.

The modification described here introduces a review gate between the external mutation source (the sheet) and the operational principal registry.

Design constraint: non-destructive staging

The central constraint is that a proposed roster change must not activate until a credentialed reviewer has approved it. The existing import path presented a structural obstacle: `db_insert_user()` is an INSERT-only operation; modifications require raw UPDATE SQL. Neither path supports a pending state. Staging proposals in a separate relation (`user_proposals`) solves this without touching `edc_users` or requiring a schema migration to the existing user table.

An analogous staging relation (`dsl_rule_proposals`) was implemented for data-scientist-initiated validation rule proposals. The two staging relations are independent; their approval functions operate on separate queues.

Schema: `user_proposals`

```
CREATE TABLE IF NOT EXISTS user_proposals (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

proposal_id      TEXT UNIQUE NOT NULL, -- username + compacted timestamp
username         TEXT NOT NULL,
change_type      TEXT NOT NULL,        -- 'NEW' or 'MODIFIED'
proposed_full_name TEXT,
proposed_email   TEXT,
proposed_role    TEXT,
proposed_site_id TEXT,
proposed_active  INTEGER,
current_role     TEXT,                  -- snapshot at staging; NULL for NEW
current_email    TEXT,
sheet_id        TEXT NOT NULL,
staged_by       TEXT NOT NULL,
staged_at       TEXT NOT NULL,
status          TEXT NOT NULL DEFAULT 'PENDING',
reviewed_by     TEXT,
reviewed_at     TEXT,
review_comments  TEXT
);

```

Schema: dsl_rule_proposals

```

CREATE TABLE IF NOT EXISTS dsl_rule_proposals (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  proposal_id TEXT UNIQUE NOT NULL, -- rule_id + compacted timestamp
  rule_id     TEXT NOT NULL,
  change_type TEXT NOT NULL,        -- 'NEW', 'MODIFIED', 'DELETED'
  proposed_dsl TEXT,
  current_dsl  TEXT,                  -- snapshot at staging; NULL for NEW
  field_code   TEXT,
  form_code    TEXT,
  rule_name    TEXT,
  error_message TEXT,
  severity     TEXT,
  rule_category TEXT,
  sheet_id     TEXT NOT NULL,
  staged_by    TEXT NOT NULL,
  staged_at    TEXT NOT NULL,
  status       TEXT NOT NULL DEFAULT 'PENDING',
  reviewed_by  TEXT,
  reviewed_at  TEXT,
  review_comments TEXT
);

```

Change detection

diff_gsheets_users() performs a read-side diff against edc_users:

1. Fetches the Study_Users tab via googlesheets4::read_sheet().
2. Queries SELECT username, role, email FROM edc_users.

3. Classifies each sheet row as **NEW** (username absent from DB) or **MODIFIED** (username present but role or email differs). Deletions are flagged in log output but never auto-staged; removing a principal from a live trial database requires explicit data-scientist action.

The analogous `diff_gsheets_rules()` diffs the `validation_rules` tab against `dsl_validation_rules` using DSL content equality.

Staging and idempotency

Both staging functions (`stage_gsheets_user_proposals()`, `stage_gsheets_proposals()`) are idempotent with respect to content. If a **PENDING** proposal already exists for the same key (username or rule_id) with identical proposed values, the row is not duplicated. If the proposed values have changed since the last poll (a coordinator revised their edit before the manager reviewed it), the existing **PENDING** row is updated in place. This prevents proposal accumulation across poll intervals while ensuring the queue always reflects the coordinator's current intent.

Approval and database rebuild

`approve_user_proposals()` for each approved `proposal_id`:

1. Validates `status = 'PENDING'`.
2. For **NEW**: calls `db_insert_user()` with the proposed values and a cryptographically random initial password (12-character alphanumeric, hashed with the system salt). The account holder must change this password on first login.
3. For **MODIFIED**: issues an `UPDATE` to `edc_users` using `COALESCE` to apply only the fields present in the proposal.
4. Marks the proposal **'APPROVED'** and records `reviewed_by`, `reviewed_at`, and any comments.

`approve_proposals()` (for DSL rules) additionally recompiles the DSL expression to R and SQL validator expressions before writing to `dsl_validation_rules`.

Both reject functions mark the proposal **'REJECTED'** and leave the live relation untouched.

Notification architecture

The notifier is a dependency-injected function passed as the `notifier` argument of `poll_gsheets_and_stage()`. Signature: `function(proposals_df)`. The default implementation (`default_proposal_notifier()`) appends a plain-text summary to a log file resolved from `ZZEDC_PROPOSAL_LOG`. A production deployment substitutes a `blastula`-based SMTP function with no changes to the detection and staging logic.

Cron integration and multi-workbook polling

Because each site has its own workbook, the poll script iterates over four sheet IDs, one per site, resolved from environment variables (`PRAZ_SHEET_UCSD`, `PRAZ_SHEET_UCLA`, `PRAZ_SHEET_UCSF`, `PRAZ_SHEET_STANFORD`). The function returns `NULL` silently when no changes are detected, producing no log output on no-op runs.

Audit trail

All state transitions – staging, approval, rejection – are recorded. User proposal transitions are recorded in `user_proposals` itself (the `reviewed_by`, `reviewed_at`, `review_comments` columns). DSL

rule transitions additionally write to `dsl_rule_change_log` via `log_dsl_rule_action()`, which captures old and new DSL values and the reviewer’s identity.

Part 2: Permissions Hierarchy – Technical Analysis (Database Expert Register)

Standard tier structure for a multi-site clinical EDC

A well-governed EDC for a project of this size (200 subjects, 4 sites) conventionally defines four tiers.

Tier 0: System administrator (technical root) Operates below the application layer. Has filesystem access to the `.db` file and can bypass all application-layer permission checks. Is not registered as an application user. Can create additional Tier 0 accounts. Is never used for day-to-day study operations. Corresponds to the OS account running the Shiny server process and the account that can `ssh` to the server.

Tier 1: Study administrator (functional root within the project) One or two named application accounts. Typically the PI and a backup. Capabilities: create or deactivate any user at any tier; grant roles up to but not exceeding their own (non-escalation invariant); modify the data dictionary and validation rules; approve any pending proposal; view all data across all sites. In REDCap terms: holds `design`, `user_rights`, `data_access_groups`, and full export rights.

Tier 2: Study manager (coordinating centre) Best practice distinguishes two sub-roles that may be merged into one account or kept separate depending on the trial’s governance structure:

- *Configuration manager*: can author and submit changes to the data dictionary and validation rules (`design` in REDCap); can view (not write) the full user roster; cannot create accounts at Tier 1; cannot grant `user_rights` to others.
- *User administrator*: can create and deactivate accounts at Tier 3 and below; cannot modify the data dictionary or validation rules.

The key invariant: a Tier 2 account cannot promote another account to Tier 1. The non-escalation rule must be enforced at the application layer, not by convention.

Tier 3: Site coordinator Data entry for their assigned site only. Cannot view other sites’ data. Can propose changes to their site’s user roster (via Google Sheets). Cannot modify any configuration directly.

Tier 4 (read-only): Monitor, sponsor, auditor Read-only access. No write permissions of any kind.

REDCap’s permission model at this project scale

REDCap separates permissions into two orthogonal axes: project-level rights and per-instrument access levels. The project-level rights most relevant to this analysis are:

Right	Scope
<code>design</code>	Modify instruments, data dictionary, branching logic, validation
<code>user_rights</code>	Add, modify, or remove project user accounts

Right	Scope
data_access_groups	Create and assign Data Access Groups (site isolation)
data_export_tool	Export records; can be restricted to de-identified
lock_record	Lock and unlock completed records
record_create, record_delete	Record lifecycle management
alerts	Configure automated notifications

REDCap enforces non-escalation: a user with `user_rights` can only grant permissions up to (not exceeding) their own. The system administrator (`superadmin`) is a separate technical account outside all projects.

For a 4-site 200-subject trial, the standard REDCap account matrix is:

Account	design	user_rights	DAG	Export	Notes
Superadmin	system	system	–	–	Tier 0; not a project account
PI	yes	yes	all	full	Tier 1
CC data manager	yes	no	all	full	Tier 2 config
CC user admin	no	yes (Tier 3 only)	all	full	Tier 2 user admin
Site coordinator	no	no	own site	none or de-id	Tier 3
Monitor	no	no	all	de-id	Tier 4

Current ZZedc role model and identified gaps

The default `dsl_rule_permissions` table defines five roles:

Role	can_view	can_create	can_edit	can_delete	can_approve	can_activate
admin	1	1	1	1	1	1
pi	1	0	0	0	1	1
data_manager	1	1	1	0	0	1
coordinator	1	0	0	0	0	0
monitor	1	0	0	0	0	0

Four gaps relative to the standard model:

Gap 1: No study_manager role. There is no role that combines configuration-write access (`can_create`, `can_edit` on dictionary and rules) with user-roster approval authority (`can_approve_users`). The coordinating centre study manager who maintains the data dictionary and approves user proposals has no appropriate role. The closest existing role is

`data_manager`, which has `can_approve = 0` (cannot approve DSL rules, and the proposal approval functions do not yet check a permission flag for user proposals).

Gap 2: No non-escalation enforcement. The user management UI offers Admin, PI, Coordinator, Data Manager, and Monitor as role choices with no application-layer check preventing a `data_manager` from promoting a `coordinator` to `admin`. REDCap enforces this at every role-assignment call.

Gap 3: pi cannot create or edit rules directly. `can_create = 0` and `can_edit = 0` for the `pi` role means the PI can approve rules but cannot author them. This is defensible (it enforces separation of authoring and approval) but is not consistent with the Tier 1 definition above and may surprise a PI who expects full access.

Gap 4: Tier 0 / application admin conflation. The technical guide documents `psmith` (the PI) as the top-level application account, conflating the study PI role with the system administrator role. A production deployment should have a dedicated `sysadmin` application account used only for system-level operations, with `psmith` registered as PI rather than Admin.

Part 3: Constructing the Permissions Hierarchy via Google Sheets

The Study_Users workbook as the authoritative seed source

The `Study_Users` tab is the principal registry seed: it defines every application user before the database is initialised. Getting the role assignments correct in the workbook before the first import is cheaper than correcting them after go-live, because post-import role changes require direct database writes or admin UI operations rather than a re-import.

The construction process is therefore:

1. Define the full desired hierarchy in the `Study_Users` tab of the coordinating centre workbook.
2. Review it against the tier model before importing.
3. Import once with `setup_zzedc_from_gsheets()`.
4. Verify each account's effective permissions with a dry-run login.
5. Add the `study_manager` role and non-escalation enforcement to `dsl_rule_permissions` as a post-import migration.

Recommended account matrix for PRAZ-L-2026

The following `Study_Users` tab structure is recommended. The coordinating centre workbook holds all non-coordinator accounts; each site workbook holds only that site's coordinator accounts.

Coordinating centre workbook (`Study_Users` tab):

username	full_name	role	site_id	active	Notes
sysadmin	System Administrator	Admin	ALL	TRUE	Tier 0/1 technical account; no study operations

username	full_name	role	site_id	active	Notes
psmith	Professor Smith	PI	ALL	TRUE	Tier 1; approves proposals, cross-site read Tier 2; dictionary + user approval authority Tier 4; read-only
cc_manager	CC Study Manager	StudyManager	ALL	TRUE	
cc_monitor	CC Monitor	Monitor	ALL	TRUE	

Per-site workbooks (Study_Users tab, one per site):

username	full_name	role	site_id	active	Notes
ucsd_coord	UCSD Coordinator	Coordinator	UCSD	TRUE	Tier 3
ucla_coord	UCLA Coordinator	Coordinator	UCLA	TRUE	Tier 3
ucsf_coord	UCSF Coordinator	UCSF	UCSF	TRUE	Tier 3
stanford_coord	Stanford Coordinator	Coordinator	Stanford	TRUE	Tier 3

Post-import: adding the StudyManager role

The `StudyManager` role does not exist in the default `ZZedc` schema. After the first import, run the following to add it and set the appropriate permissions. This must be run by the data scientist (Tier 0) before the `cc_manager` account can use approval functions:

```
library(zzedc)
library(DBI)

con <- connect_encrypted_db(db_path = 'data/praz-large.db')

# Register the new role in the DSL permissions table.
dbExecute(con, "
  INSERT OR IGNORE INTO dsl_rule_permissions
    (role_name, can_view, can_create, can_edit,
     can_delete, can_approve, can_activate)
  VALUES ('studymanager', 1, 1, 1, 0, 1, 1)
")

dbDisconnect(con)
```

This gives the `StudyManager` role full configuration-write and approval authority over DSL rules. User-proposal approval authority for `approve_user_proposals()` is enforced by role membership

in `edc_users` rather than `dsl_rule_permissions`; extend the check in `approve_user_proposals()` to verify role IN ('admin', 'pi', 'studymanager') once the function is updated.

Post-import: enforcing non-escalation in the user management UI

Add a role ceiling check to the user management module server. The principle: a user can only assign roles that appear below their own tier in the hierarchy. A concrete implementation:

```
ROLE_TIER <- c(
  admin      = 1L,
  pi         = 1L,
  studymanager = 2L,
  data_manager = 2L,
  coordinator = 3L,
  monitor    = 4L
)

assignable_roles <- function(actor_role) {
  actor_tier <- ROLE_TIER[tolower(actor_role)]
  names(ROLE_TIER)[ROLE_TIER > actor_tier]
}
```

Pass `assignable_roles(session_user_role)` as the `choices` vector when rendering the role select input in the user management UI. The PI and admin accounts see all roles; the study manager sees only coordinator and monitor.

Separating sysadmin from PI in the configuration file

Update `zzedc_config.yml` to distinguish the technical admin account from the PI:

```
admin:
  username: "sysadmin"
  fullname: "System Administrator"
  email: "sysadmin@university.edu"
  password: "ChangeToVaultManagedSecret!"

study:
  pi_username: "psmith"
  pi_name: "Professor Smith"
  pi_email: "psmith@university.edu"
```

The `sysadmin` account is used only for system initialisation, database migrations, and emergency recovery. It should not be used for day-to-day study operations. Its password should be stored in an institutional secrets vault (not in the config file under version control) and rotated after initial setup.

Part 4: Change Monitor – Plain-Language Description (Data Scientist Register)

What was the problem?

Coordinators at each recruitment site have editor access to the `Study_Users` tab of their site's Google Sheets workbook. This lets them propose new user accounts or request role changes for their site team. Before the change monitor was built, the only way to get those edits into the `ZZedc` database was for you to run a manual re-import. There was no checkpoint: the change went live immediately, whether or not it was correct.

For a running clinical trial, that is not safe. A coordinator could accidentally assign the wrong role to an account, or add a user with an incorrect site scope, and that account would have incorrect access from the moment you ran the import.

What was built

Three pieces were added:

- 1. A staging table in the database.** When the monitor detects a coordinator's edit in the `Study_Users` sheet, it does not change the live user roster. It writes the proposed change to a separate holding table called `user_proposals`. Think of it as an inbox: proposals sit there until the manager acts on them.
- 2. A polling script and a notifier.** A script runs on the study server every 15 minutes. It reads each site's workbook, compares it with the current user roster, and for any row that has changed, writes a proposal and triggers a notification. By default the notification is a plain-text log file. For production use you supply a short function that sends an email (a `blastula` example is in the technical guide, Section 7a.3).
- 3. Approval and rejection commands.** When the manager receives the notification, they run:
 - `get_pending_user_proposals()` to see the queue.
 - `approve_user_proposals()` to accept one or more proposals. New accounts are created immediately with a temporary password; the account holder must change it on first login.
 - `reject_user_proposals()` to decline a proposal with a recorded reason.

What you need to do to set this up

1. Fill in the `Study_Users` tab of each workbook using the account matrix in Part 3 of this document.
2. Import the coordinating centre workbook first: `setup_zzedc_from_gsheets(sheet_url = CC_URL, db_path = ...)`.
3. Import each site workbook: `setup_zzedc_from_gsheets(sheet_url = SITE_URL, db_path = ...)`.
4. Run the post-import SQL in Part 3 to add the `StudyManager` role.
5. Create `poll_users.R` and add the cron entry per Section 7a.2 of the technical guide.
6. Set the four `PRAZ_SHEET_*` environment variables and the SMTP variables on the server.

What the standard permission hierarchy means for day-to-day operations

- **sysadmin:** used only by you (the data scientist) for initialisation and emergency operations. Never log in as `sysadmin` to enter or review study data.

- **psmith (PI)**: approves pending user and rule proposals; views cross-site data; does not routinely change database configuration.
- **cc_manager (study manager)**: maintains the data dictionary and validation rules; approves or rejects coordinator user proposals; cannot create admin or PI accounts.
- **Site coordinators**: data entry only; can propose new accounts for their site via the Google Sheet.
- If the PI wants to add a second study manager, add a new row to the coordinating centre Study_Users tab with role = StudyManager and put the proposal through the normal approval workflow.

Files modified or created

File	Change
R/gsheets_monitor.R	New file; 10 exported functions covering both user and DSL rule proposals
NAMESPACE	10 new <code>export()</code> entries
vignettes/prazosin-large/prazosin-large-technical-guide.Rmd	Sections 7.3 (corrected access table), 7a (5 subsections, updated for Study_Users monitoring), 2 troubleshooting entries
vignettes/prazosin-large/prazosin-large-user-guide.Rmd	Section 9 (corrected access model, approval-loop description), 2 troubleshooting entries, 3 glossary entries
docs/gsheets-monitor-design.md	This document

Rendered on 2026-05-28 at 18:37 PDT. Source: ~/prj/sfw/05-zzedc/zzedc/docs/gsheets-monitor-design.md